Advanced Mockito Hands-On Exercises

Exercise 1: Mocking Databases and Repositories

Explanation

This exercise tests a service that depends on a database repository. Mockito creates a mock Repository instead of connecting to a real database. The repository method is stubbed to return predefined data, and the test verifies that the service processes it correctly.

Source Code

Repository.java

```
package com.cognizant.advancedmockito;

public interface Repository {
    String getData();
}
```

Service.java

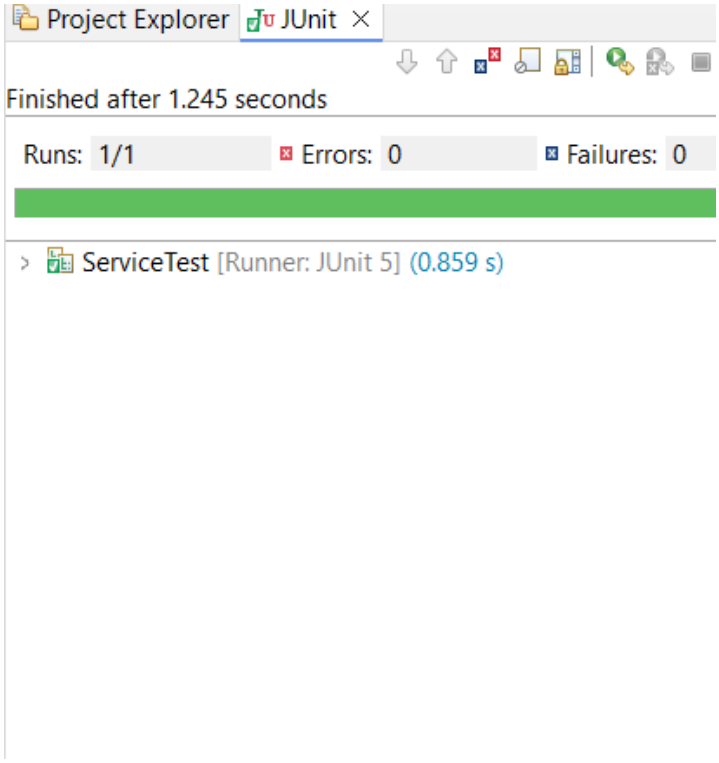
```
package com.cognizant.advancedmockito;

public class Service {
    private Repository repository;

    public Service(Repository repository) {
        this.repository = repository;
    }

    public String processData() {
        return "Processed " + repository.getData();
    }
}
```

ServiceTest.java

```
package com.cognizant.advancedmockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;

public class ServiceTest {
    @Test
    public void testServiceWithMockRepository() {
        Repository mockRepository = mock(Repository.class);
        when(mockRepository.getData()).thenReturn("Mock Data");

        Service service = new Service(mockRepository);
        String result = service.processData();

        assertEquals("Processed Mock Data", result);
    }
}
```

Output

The JUnit test completed successfully with 1/1 run, 0 errors and 0 failures. The green bar confirms that the test passed.



Exercise 2: Mocking External Services (RESTful APIs)

Explanation

This exercise tests a service that calls an external RESTful API. A mock RestClient avoids a real network request. Mockito returns a predefined response, and the test verifies that ApiService handles the response correctly.

Source Code

RestClient.java

```
package com.cognizant.advancedmockito;

public interface RestClient {
    String getResponse();
}
```

ApiService.java

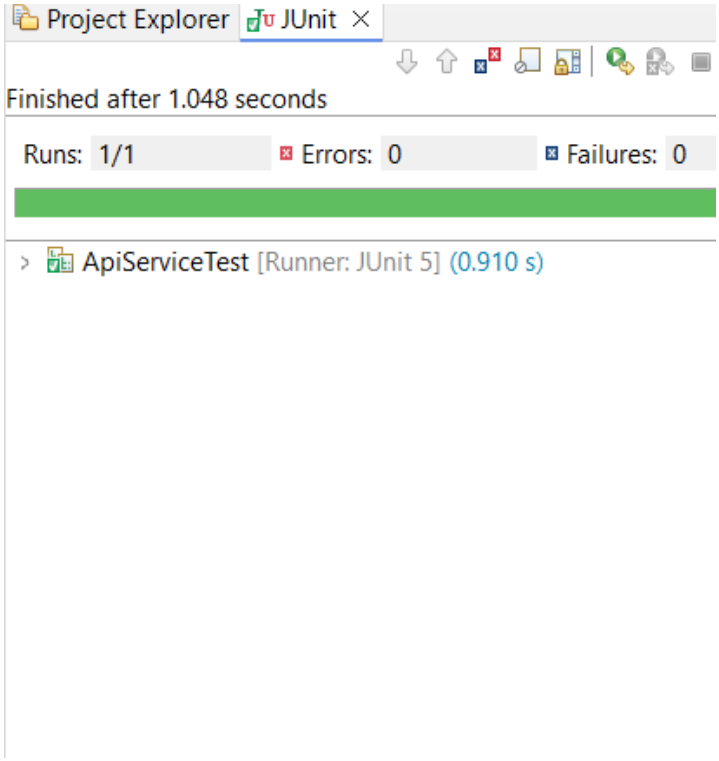
```
package com.cognizant.advancedmockito;

public class ApiService {
    private RestClient restClient;

    public ApiService(RestClient restClient) {
        this.restClient = restClient;
    }

    public String fetchData() {
        return "Fetched " + restClient.getResponse();
    }
}
```

ApiServiceTest.java

```
package com.cognizant.advancedmockito;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class ApiServiceTest {
    @Test
    public void testServiceWithMockRestClient() {
        RestClient mockRestClient = mock(RestClient.class);
        when(mockRestClient.getResponse()).thenReturn("Mock Response");

        ApiService apiService = new ApiService(mockRestClient);
        String result = apiService.fetchData();

        assertEquals("Fetched Mock Response", result);
    }
}
```

Output

The JUnit test completed successfully with 1/1 run, 0 errors and 0 failures. The green bar confirms that the test passed.


Project Explorer
JUnit
Finished after 1.048 seconds
Runs: 1/1
Errors: 0
Failures: 0
ApiServiceTest [Runner: JUnit 5] (0.910 s)

Exercise 3: Mocking File I/O

Explanation

This exercise tests file processing without accessing real files. Mock FileReader and FileWriter objects simulate file operations. The test checks the processed result and verifies that the expected content is sent to the writer.

Source Code

FileReader.java

```
package com.cognizant.advancedmockito;

public interface FileReader {
    String read();
}
```

FileWriter.java

```
package com.cognizant.advancedmockito;

public interface FileWriter {
    void write(String content);
}
```

FileService.java

```
package com.cognizant.advancedmockito;

public class FileService {
    private FileReader fileReader;
    private FileWriter fileWriter;

    public FileService(FileReader fileReader, FileWriter fileWriter) {
        this.fileReader = fileReader;
        this.fileWriter = fileWriter;
    }

    public String processFile() {
        String result = "Processed " + fileReader.read();
        fileWriter.write(result);
        return result;
    }
}
```

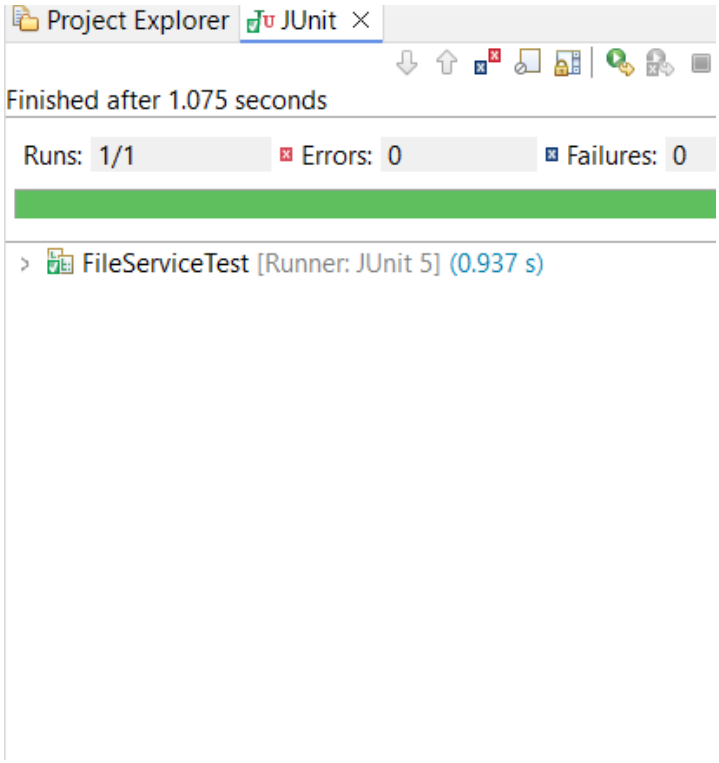
FileServiceTest.java

```
package com.cognizant.advancedmockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;

public class FileServiceTest {
    @Test
    public void testServiceWithMockFileIO() {
        FileReader mockFileReader = mock(FileReader.class);
        FileWriter mockFileWriter = mock(FileWriter.class);

        when(mockFileReader.read()).thenReturn("Mock File Content");
```

```
        FileService fileService =
            new FileService(mockFileReader, mockFileWriter);

        String result = fileService.processFile();

        assertEquals("Processed Mock File Content", result);
        verify(mockFileWriter).write("Processed Mock File Content");
    }
}
```

Output

The JUnit test completed successfully with 1/1 run, 0 errors and 0 failures. The green bar confirms that the test passed.



Exercise 4: Mocking Network Interactions

Explanation

This exercise tests a service that communicates with a network resource. A mock NetworkClient replaces the real connection and returns a fixed response. The test verifies the service result without contacting a real server.

Source Code

NetworkClient.java

```
package com.cognizant.advancedmockito;

public interface NetworkClient {
    String connect();
}
```

NetworkService.java

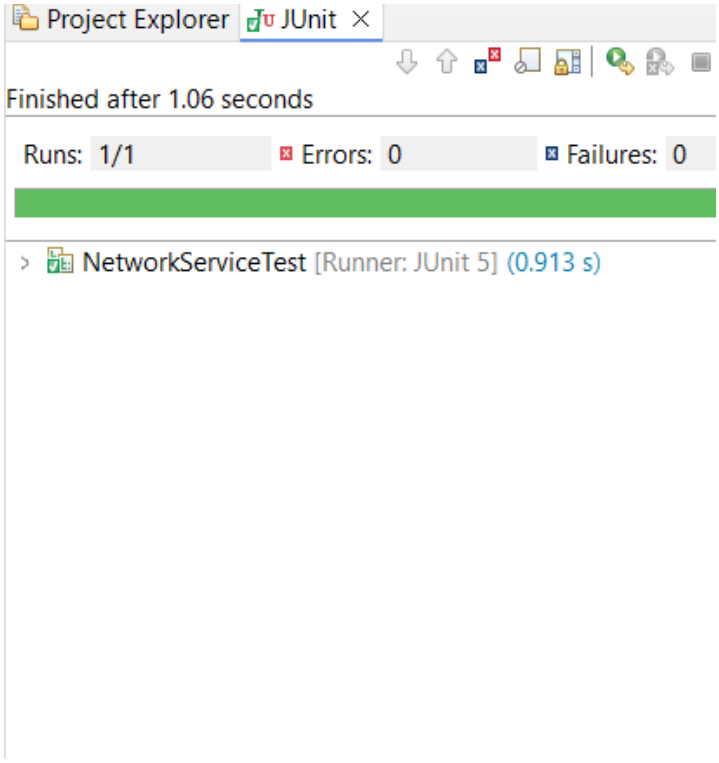
```
package com.cognizant.advancedmockito;

public class NetworkService {
    private NetworkClient networkClient;

    public NetworkService(NetworkClient networkClient) {
        this.networkClient = networkClient;
    }

    public String connectToServer() {
        return "Connected to " + networkClient.connect();
    }
}
```

NetworkServiceTest.java

```
package com.cognizant.advancedmockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;

public class NetworkServiceTest {
    @Test
    public void testServiceWithMockNetworkClient() {
        NetworkClient mockNetworkClient = mock(NetworkClient.class);

        when(mockNetworkClient.connect()).thenReturn("Mock Connection");

        NetworkService networkService =
            new NetworkService(mockNetworkClient);

        String result = networkService.connectToServer();

        assertEquals("Connected to Mock Connection", result);
    }
}
```

Output

The JUnit test completed successfully with 1/1 run, 0 errors and 0 failures. The green bar confirms that the test passed.



Exercise 5: Mocking Multiple Return Values

Explanation

This exercise demonstrates consecutive return values. The same mocked repository method is called twice, but Mockito returns different data for each call. The test verifies that both returned values are processed correctly.

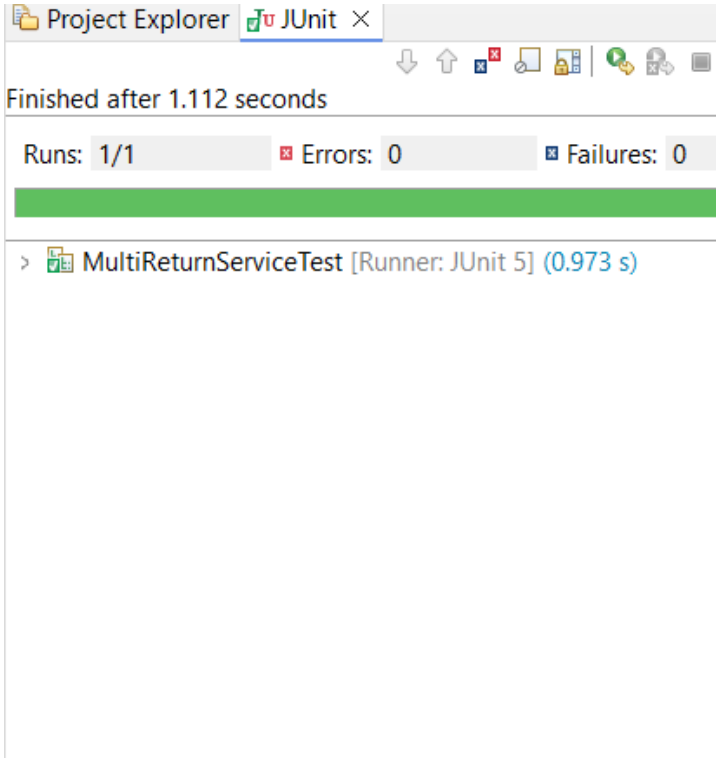
Source Code

MultiReturnServiceTest.java

```
package com.cognizant.advancedmockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;

public class MultiReturnServiceTest {
    @Test
    public void testServiceWithMultipleReturnValues() {
        Repository mockRepository = mock(Repository.class);

        when(mockRepository.getData())
            .thenReturn("First Mock Data")
            .thenReturn("Second Mock Data");

        Service service = new Service(mockRepository);

        String firstResult = service.processData();
        String secondResult = service.processData();

        assertEquals("Processed First Mock Data", firstResult);
        assertEquals("Processed Second Mock Data", secondResult);
    }
}
```

Output

The JUnit test completed successfully with 1/1 run, 0 errors and 0 failures. The green bar confirms that the test passed.


Project Explorer
JUnit
Finished after 1.112 seconds
Runs: 1/1
Errors: 0
Failures: 0
MultiReturnServiceTest [Runner: JUnit 5] (0.973 s)